



Список тренингов / Промпт-инжиниринг / Промпт-инжиниринг 2.0 / Модуль 9. Оптимизация затрат на ИИ. Стратегии развертывания ИИ-моделей (/teach/control/stream/view/id/931478150)

РЕ9.2 Локальное развертывание моделей

Предыдущий урок

(/pl/teach/control/lesson/view?id=338667559)

Следующий урок

(/pl/teach/control/lesson/view?id=338667562)

Дорогой студент! Интерфейсы современных приложений могут очень быстро меняться. Если вдруг то, что видишь ты, отличается от того, что демонстрирует эксперт, настолько, что тебе непонятно, куда нажимать — напиши об этом в учебный чат, указав номер урока и видео. Наш куратор обязательно поможет тебе разобраться, а мы постараемся как можно скорее актуализировать материал :)

Уважаемые студенты!

В рамках данного урока могут рассматриваться примеры передачи персональных данных при регистрации на иностранных сервисах, а также обработки и хранения пользовательских данных.

В соответствии с требованиями Федерального закона № 152-ФЗ «О персональных данных», обращаем ваше внимание: в демонстрационных примерах и при выполнении домашних заданий необходимо использовать исключительно тестовые
Действия

данные или тестовые аккаунты, в которых не указаны персональные данные — используйте вымышленные ники, дополнительные данные и т.п.

Использование иностранных сервисов не является обязательным. Вы вправе выполнять домашние задания на аналогичных разрешённых платформах по вашему выбору.

Результат дня

Узнаешь, как выбирать и оценивать GPU для запуска моделей, что такое эмбединги и как их использовать, а также какие инструменты применять для локального развертывания моделей.

✚ Данный урок в большей степени ориентирован на знакомство с теорией. Некоторые темы из этого урока мы будем разбирать подробнее на следующих занятиях.

Теория на сегодня

- Как выбрать GPU и оценить какие ресурсы потребуются для запуска LLM?
- Что такое эмбединги и как их можно использовать?
- Какими тулами можно пользоваться, чтобы запускать модели локально?

Время изучения материалов

▶ Время чтения и просмотра: ~1 час

🕒 Время выполнения задания: ~1 час

1. GPU, веса и квантизация

Я — Иван Родин

- Постдок-исследователь, Университет Катании
- Компьютерное зрение, FPV, Активные роботы
- Исследования в области ИИ
- Публикации в научных журналах

LinkedIn: [https://www.linkedin.com/in/ivanrodin](#)
TG-Channel: [https://t.me/rodin_ivan](#)





Вы остановились на 00:33

ПРОДОЛЖИТЬ СНАЧАЛА

zerocoder.ru



В этом уроке мы обсудим локальное развертывание моделей. Мы разберемся, как запускать модели и на что следует обращать внимание.

Действия

GPU, веса и квантизация — как оценить системные требования

Начнем с обсуждения видеокарт и оценки системных требований и рассмотрим важные параметры видеокарты:

- Объем памяти VRAM. Для запуска локальных моделей желательно иметь хотя бы 12 гигабайт памяти на видеокарте;
- Количество ядер CUDA/Stream Processors. Хорошо бы иметь хотя бы 4000 ядер, но здесь, как и с памятью, чем больше, тем лучше;
- Пропускная способность видеокарты (Memory bandwidth), отвечающую за скорость обработки данных — она должна быть не менее 600 Гб/с;
- Энергопотребление, что важно, если в планах создание большого сервера из нескольких видеокарт и если стоимость электроэнергии критична.

Это важно для пользователей, планирующих собрать суперкомпьютер или обучить новую версию GPT с нуля, поскольку энергопотребление будет существенно влиять на стоимость разработки.

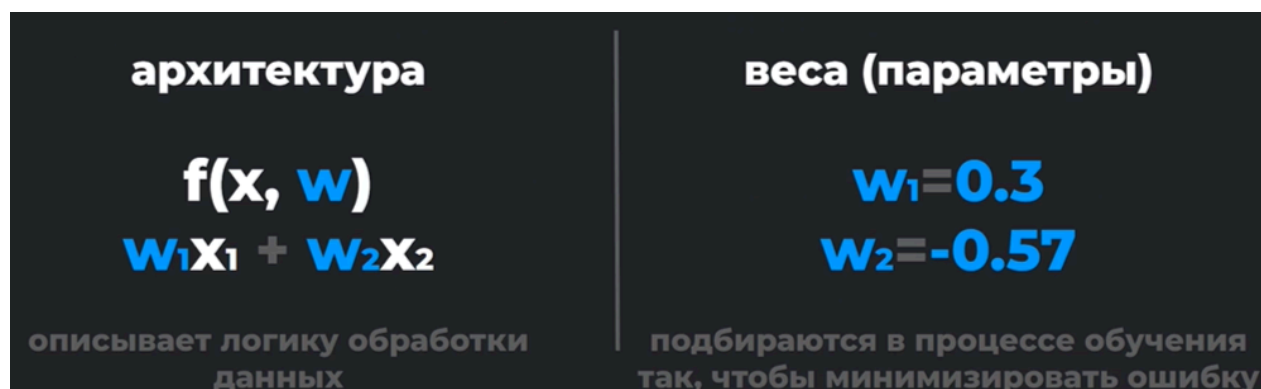
Сравнение GPU

GPU Model	VRAM	Memory Bandwidth	CUDA Cores / Stream Processors	Tensor Cores
NVIDIA A100	80GB	2TB/s	6912 CUDA cores	432
NVIDIA H100	80GB	3TB/s	14592 CUDA cores	1152
NVIDIA RTX A6000	48GB	768GB/s	10752 CUDA cores	336
NVIDIA RTX 4090	24GB	1008GB/s	16384 CUDA cores	512
NVIDIA RTX 3090 Ti	24GB	1008GB/s	10752 CUDA cores	336
AMD Radeon RX 7900 XTX	24GB	960GB/s	6144 Stream Processors	N/A

Сравнивая популярные модели видеокарт по различным параметрам, выделим индустриальные модели NVIDIA A100, NVIDIA H100 и NVIDIA V100. Эти видеокарты обладают большим объемом памяти и используются компаниями, предоставляющими облачные решения.

В то же время существуют персональные видеокарты, которые можно приобрести самостоятельно для установки на сервер. Их преимущество в меньшем энергопотреблении и, в некоторых случаях, в большем количестве ядер. Они подходят для небольших стартапов или личного использования, когда видеокарта требуется для периодического запуска задач.

Как понять, какую видеокарту использовать для конкретной модели? Нужно учитывать требования и объем памяти. Для этого вспоминаем, как представляется модель. Любая модель глубокого обучения состоит из архитектуры и весов. Архитектура описывает логику обработки данных, а веса — это параметры модели, подбираемые в процессе обучения для минимизации ошибки.

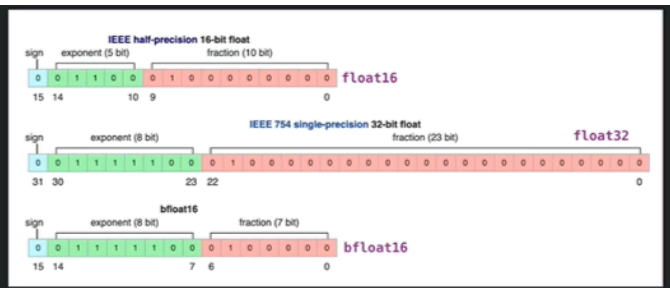


Количество параметров модели определяет ее размер и объем занимаемой памяти. Современные большие языковые модели имеют миллиарды параметров. Например, GPT-3 имеет 175 миллиардов параметров.

Как оценить объем GPU

Объем видеокарты рассчитывается, исходя из количества параметров и формата хранения весов. Наиболее распространенный формат хранения — FP32, занимающий в памяти компьютера 4 байта на параметр. Таким образом можно вычислить примерный необходимый объем памяти, зная количество параметров модели.

- **FP32 (float32) = 4 байта**
- **BF16 (bfloat16) / FP16 (float16) = 2 байта**
- **INT8 = 1 байт**
- **INT4 = 0.5 байта**



Многие модели используют другие форматы, такие как BF16 или FP16, которые занимают 2 байта на параметр. Иногда веса хранятся в целочисленном формате int8 или int4, занимая 1 байт или даже полбайта на параметр.

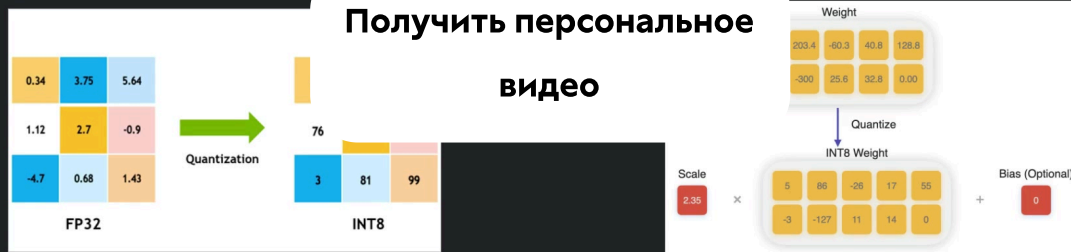
2. Квантизация

Квантизация

Часто модели обучают с высокой точностью (fp32), а затем **квантизуют** - уменьшают объем с минимальной потерей качества



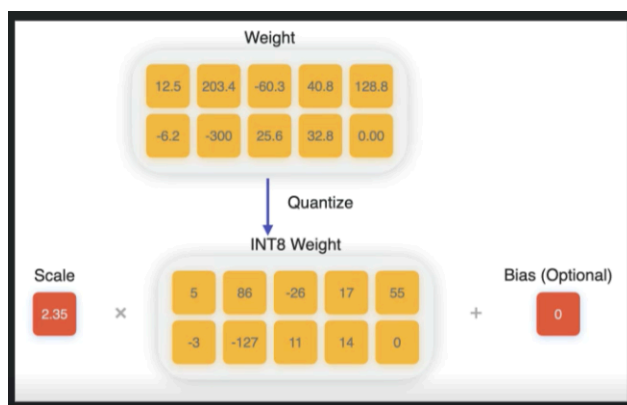
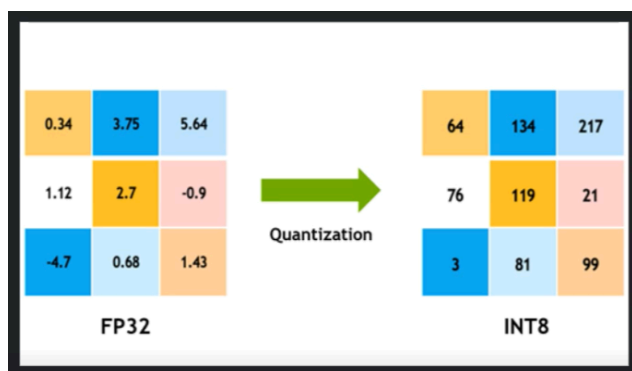
Получить персональное видео



zerocoder.ru

Еще один важный концепт, который помогает нам запускать модели локально — это квантизация.

✓ Квантизация — это уменьшение объема модели с минимальной потерей качества. При квантизации берется модель, у которой изначально веса хранятся в формате FP32, и веса модели переводятся в новый формат, например, формат INT8, который занимает в памяти компьютера в четыре раза меньше места.



Квантизация — это очень полезная технология. Выполнить квантизацию не очень просто, так как нельзя просто взять и округлить все веса модели. Это не сработает, и результаты будут очень плохими. Однако **существуют готовые библиотеки**, которые выполняют квантизацию достаточно хорошо, и есть готовые квантизованные модели, доступные на **Hugging Face**. Вы можете скачать квантизованные модели прямо оттуда.

HUDDING FACE

Конечно, квантизованные модели будут **показывать качество чуть хуже, чем исходные неквантизованные версии**. Однако это достаточно хороший компромисс, так как потеря качества здесь незначительная, но при этом заметно увеличится память и скорость работы модели. Если есть планы запускать модели локально, в этом случае квантизация и квантизованные модели очень помогут.

Обучение модели

Еще одно, что стоит обсудить в контексте локального запуска моделей, — это режим использования модели. Если мы берем готовую модель и используем ее для генерации ответов на запросы, то мы говорим про режим инференса. В этом режиме модель получает на вход **X** и генерирует **Y**. Если же мы хотим обучить или

дообучить модель на своих данных, то нужен уже другой режим использования модели. Для этого нужно обучить модель, и процесс обучения требует значительно больше ресурсов.

✅ Инференс модели — это процесс использования уже обученной модели для предсказания или генерации выходных данных на основе новых входных данных. В этом режиме модель не изменяется, она просто применяет свои заранее обученные параметры для выполнения заданий.

✅ Обучение модели — это процесс, в ходе которого модель настраивает свои параметры (веса) на основе входных данных и соответствующих правильных ответов. Цель обучения — минимизировать ошибку предсказаний модели, чтобы она могла правильно обрабатывать новые данные.

Обучение модели может требовать в пять-шесть раз больше ресурсов, чем инференс. Например, для инференса может потребоваться 10 гигабайт оперативной памяти на видеокарте, а для обучения модели уже может понадобиться 60 гигабайт. В таком случае уместить свои вычисления в память видеокарты не получится, хотя для инференса модель будет работать хорошо.

На платформе Hugging Face есть ресурс, где можно посмотреть требования к моделям:

https://huggingface.co/docs/accelerate/main/en/usage_guides/model_size_estimator

(https://huggingface.co/docs/accelerate/main/en/usage_guides/model_size_estimator)

Здесь можно оценить, сколько модель будет занимать места в памяти в режиме инференса и в режиме обучения.

Перейдем по указанной ссылке. Возьмем для примера базовую модель и посмотрим в разных форматах, сколько она будет занимать: для этого ставим галочки для разных форматов. Нажимаем **Calculate Memory Usage**.

Model Name or URL

bert-base-cased

Library

☒ auto

☐ transformers

☐ timm

Model Precision

☒ float32

☒ float16/bfloat16

☒ int8 ☒ int4

API Token

Optional (for gated models)

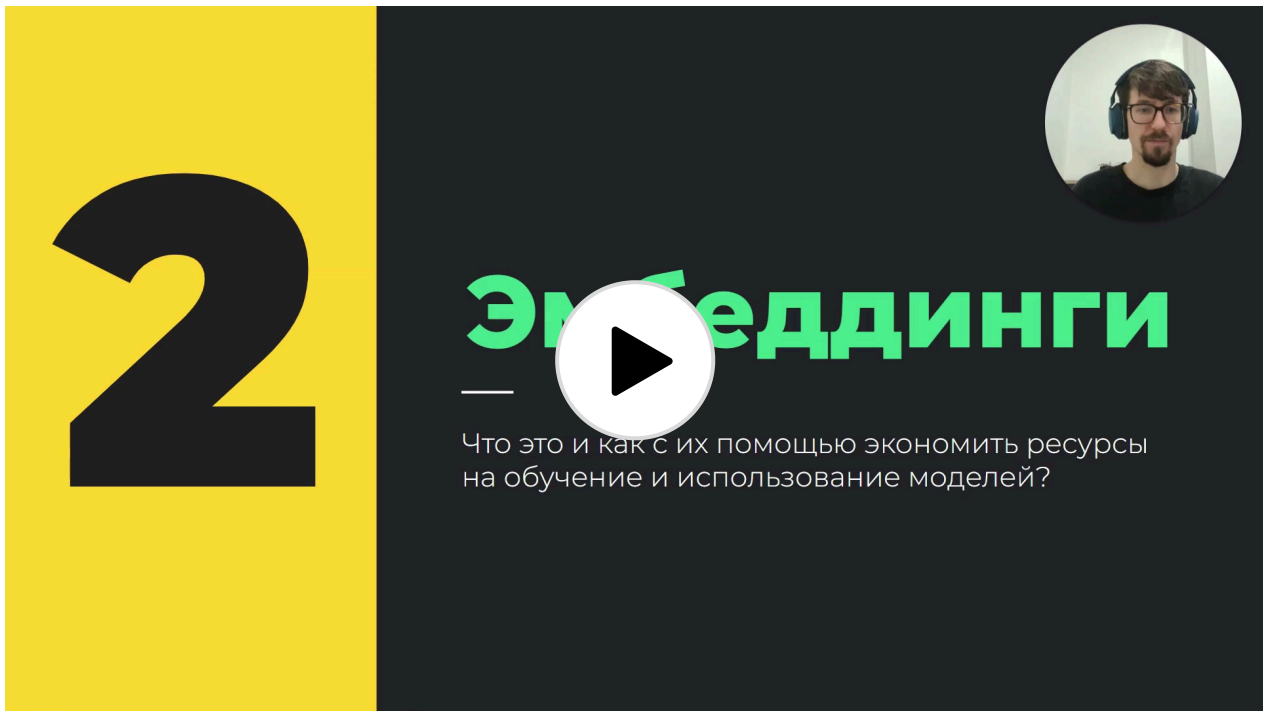
Calculate Memory Usage

В результате мы увидим информацию о том, сколько занимает самый большой слой и полный размер модели (**Total Size**). Нам нужно обращать внимание именно на этот параметр, поскольку наша задача — уместить модель в память имеющейся видеокарты. Также здесь показано, сколько места будет занимать модель, если мы будем обучать эту модель.

Например, обычный размер модели составляет 412 мегабайт, а если мы будем обучать модель (**Training using**), то понадобится примерно в четыре раза больше памяти.

Однако существуют подходы, позволяющие сократить ресурсы, требуемые для обучения модели. Один из таких подходов называется LoRA. Если возникнет необходимость дообучать модели локально, можно посмотреть, как работает LoRA и изучить это более подробно.

3. Эмбединги



Еще один важный концепт, который пригодится при работе с локальными моделями — это концепт эмбедингов. Разберемся, что это такое и как с помощью эмбедингов экономить ресурсы на использование и обучение моделей.

✅ Эмбединг — это способ представления объектов, например, текстов, в виде векторов фиксированной длины.

Другими словами, мы можем взять любой текст и преобразовать его в вектор, у которого будет определенное количество координат, обычно исчисляемое несколькими сотнями или парой тысяч. Каждая из этих координат содержит некоторое число, и весь вектор кодирует исходный текст.



Действия

Эмбе́ддинги можно строить не только для текстов, но и для данных различных модальностей: слов, изображений, видео. Концепт эмбе́ддингов используется во всем глубоком обучении, не ограничиваясь большими языковыми моделями и текстами.

Функция эмбе́ддингов

Эмбе́ддинги кодируют смысл того, что мы анализируем. Например, если мы строим эмбе́ддинги для четырех текстов: два текста про Санкт-Петербург, один про Лондон, и одно произведение Пушкина, прогоняя эти тексты через модель эмбе́ддинга, мы получаем разные векторы.

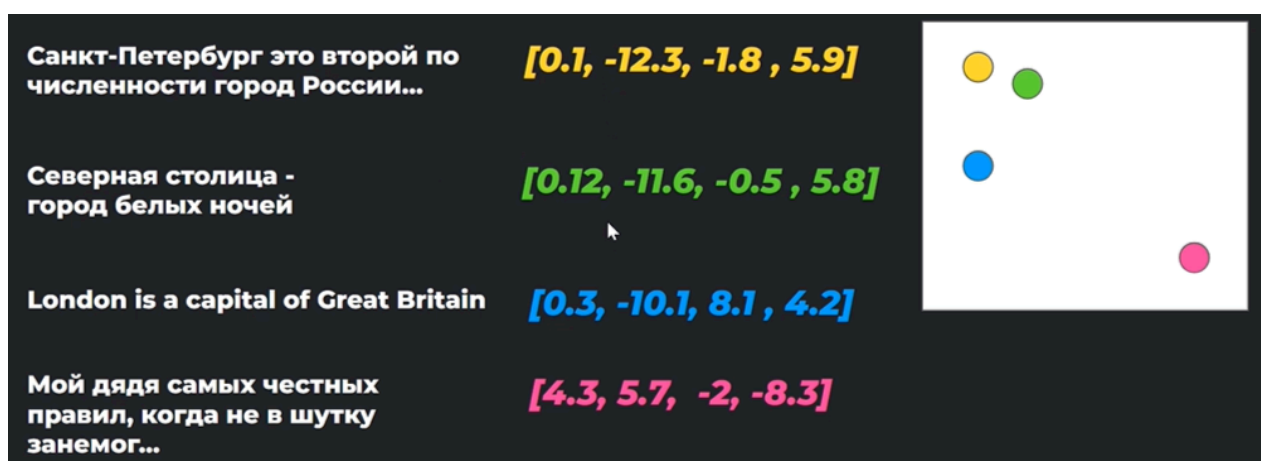
Санкт-Петербург это второй по численности город России...

Северная столица - город белых ночей

London is a capital of Great Britain

Мой дядя самых честных правил, когда не в шутку занемог...

Для наглядности изобразим не сотню координат, а четыре. Векторы, представляющие похожие тексты, будут похожи между собой. Визуализируя эти эмбе́ддинги в пространстве, мы увидим, что объекты с похожими эмбе́ддингами расположены близко друг к другу, а непохожие объекты — далеко.



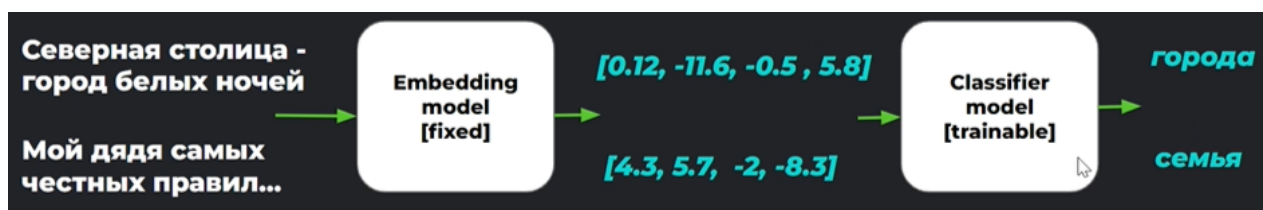
Смысл каждой координаты в эмбе́ддинге трудно интерпретировать, но, сравнивая эмбе́ддинги, мы можем понять, какие объекты похожи друг на друга. В приведенном примере мы видим, что координаты первых двух текстов похожи, так как тексты имеют схожую тематику.

Также эмбединги можно визуализировать в пространстве, и эмбединги с похожими объектами, с похожими координатами будут располагаться близко друг к другу, а не похожие объекты будут стоять в отдалении. Эмбединги можно использовать для анализа текста, определения языка, выявления тона текста и т.д.

Таким образом, эмбединг — это абстрактное понятие, помогающее сравнивать объекты и решать различные задачи.

Использование эмбедингов

Эмбединги используют при решении **вспомогательных задач**, например, классификации или кластеризации текстов, когда не требуется генерация новых текстов. Для такой задачи необязательно использовать большую языковую модель, которая будет тратить большое количество ресурса. Вместо этого можно использовать более простой инструмент: мы можем взять тексты, построить для них эмбединги и на их основе создать уже следующую модель для классификации. Модель, создающая эмбединги, фиксирована и не требует обучения. Это позволяет экономить ресурсы. Классификатор, который использует эмбединги, может быть простым алгоритмом машинного обучения, работающим быстро и эффективно.



Как получать эмбединги

- Через API. если мы хотим получить эмбединги от моделей GPT, мы можем перейти по ссылке от OpenAI и изучить документацию по ссылке:
<https://platform.openai.com/docs/guides/embeddings>
(<https://platform.openai.com/docs/guides/embeddings>)
- HuggingFace в разделе Sentence Transformers:
<https://huggingface.co/sentence-transformers>
(<https://huggingface.co/sentence-transformers>)

Работа с эмбедами через API или HuggingFace требует всего нескольких строк кода: импортируем библиотеку, задаем модель, предложение и получаем эмбеддинг.

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer('paraphrase-MiniLM-L6-v2')

# Sentences we want to encode. Example:
sentence = ['This framework generates embeddings for each input sentence']

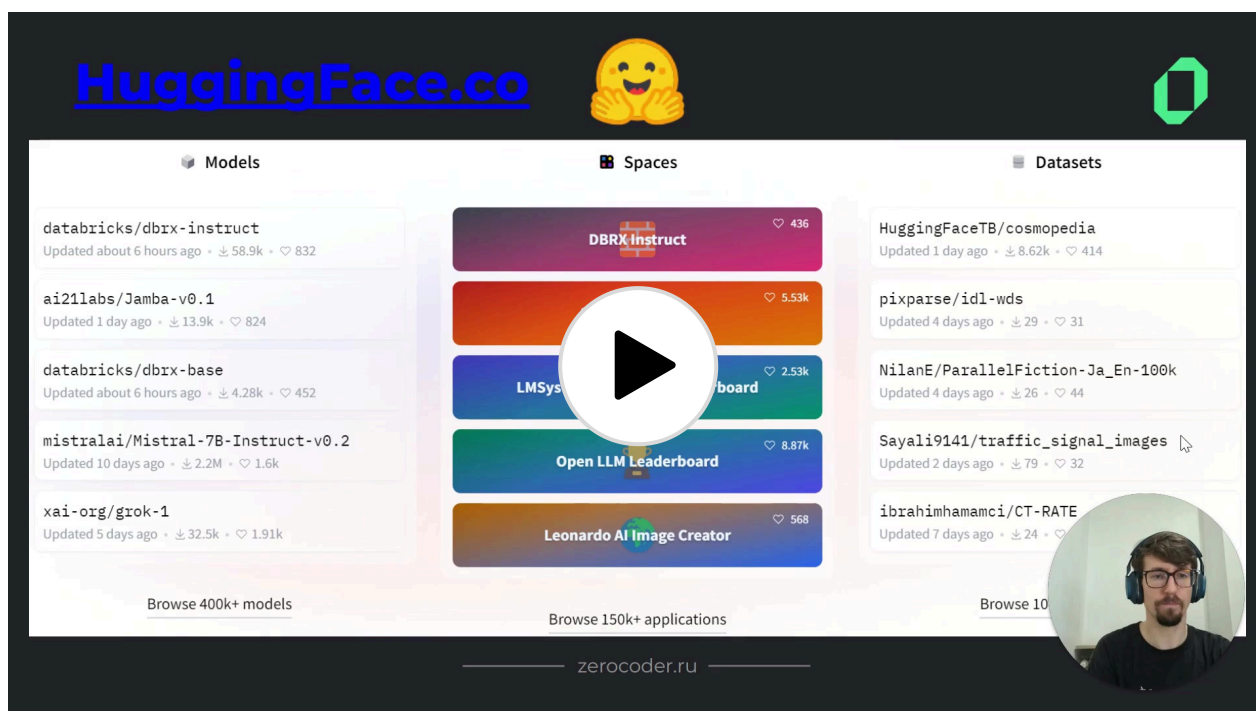
# Sentences are encoded by calling model.encode()
embedding = model.encode(sentence)
```

Когда использовать эмбединги

1. При решении простых задач на понимание текста: классификация, кластеризация;
2. При создании вспомогательных моделей: например, для оценки качества ответа или классификации темы в мультимодальной системе;
3. При создании мультимодальных систем: для работы с текстами и изображениями, где эмбединги помогут сопоставить текст и изображение одного объекта.

Таким образом, эмбединги — это полезный инструмент для решения различных задач в области анализа данных и машинного обучения.

4. Как запускать LLM локально



Обсудим, как запускать модели локально и какие инструменты для этого могут пригодиться. Также рассмотрим, как запустить модель на компьютере в режиме реального времени.

Hugging Face и библиотека Transformers

Самый полезный тул — это ресурс Hugging Face и библиотека Transformers. Hugging Face предлагает широкий набор возможностей.

Еще раз вспомним, как пользоваться Hugging Face, как искать модели.

1. Мы заходим на сайт Hugging Face.
2. Вдем во вкладку **Models**.
3. Находим интересующую нас модель.

При этом здесь можно настроить поиск с помощью фильтров. Здесь представлены не только большие языковые модели, но и множество моделей для компьютерного зрения.

Действия

4. Так как на курсе мы обсуждаем языковые модели, поэтому можем перейти во вкладку Llama.
5. Для выбора этой модели кликаем на **Use this model**.
6. Смотрим, как с помощью библиотеки **Transformers** получить и запустить модель.

Есть два варианта использования:

- в режиме pipeline, где объединены токенизатор и сама модель;
- в режиме, где можно разъединить токенизатор и модель и использовать отдельно для различных целей

Также здесь можно найти файлы конфигурации модели, конфигурацию токенизатора, веса модели, помощь сообщества и многое другое.

Hugging Face — это основной ресурс, и каждый раз, когда речь идет о запуске моделей локально, мы начинаем с поиска моделей здесь.

Однако для использования моделей через библиотеку **Transformers** нам нужно установить определенные компоненты. Прежде всего, нужно иметь **Python**, установить библиотеку **Transformers**, и желательно иметь подходящую видеокарту и драйверы для нее, которые свяжут GPU с высокоуровневыми библиотеками типа PyTorch или Transformers.

Инструмент Docker Container

Установка множества пакетов может быть сложной, особенно если нужно запускать различные модели с различными ресурсами и требованиями. Например, одна модель требует драйвер CUDA восьмой версии, другая — CUDA десятой версии. Мы не будем переустанавливать драйверы для каждой модели. Нам нужно

запускать эти модели изолированно, чтобы не создавать себе сложности с установкой и иметь возможность пользоваться разными ресурсами. Для этого нам помогает инструмент Docker Container.

✓ Docker — это платформа для упаковки, доставки и запуска приложений в контейнерах. Docker обеспечивает легкость, переносимость и консистентность на любых системах.

DOCKER CONTAINER

Идея заключается в том, что в рамках операционной системы мы можем создавать изолированные контейнеры, внутри которых будет работать другая операционная система. В рамках каждого контейнера мы можем собирать нужный набор библиотек, программ, драйверов и всего необходимого. Контейнеры не конфликтуют между собой, и у нас не будет проблем с их использованием.

Для того, чтобы углубить знания о Docker, рекомендуем посмотреть обучающий видеоурок на YouTube, который подробно разбирает принципы работы Docker:
<https://youtu.be/O8N1lvkljig?si=7PD5DqRjGUdpWyQe>
(<https://youtu.be/O8N1lvkljig?si=7PD5DqRjGUdpWyQe>)

Помимо изолированного запуска нескольких контейнеров, Docker имеет еще одно преимущество — Docker Hub. Это коллекция образов, из которых собираются контейнеры. На Docker Hub можно найти множество образов для различных приложений.

Перейдем по ссылке <https://hub.docker.com/> (<https://hub.docker.com/>) и попробуем найти нужные нам образы. Например, если для запуска приложений от Hugging Face нам нужен Python, драйверы для видеокарт и библиотеки в Python, мы можем найти образы от Hugging Face, введя в окне поиска "Hugging Face".

Как только мы начинаем вводить запрос, система сразу выдает в качестве подсказок популярные образы. Кликнем на образ, который используем в работе: Hugging Face/Transformers-pyTorch-GPU. Этот образ включает библиотеку Transformer, PyTorch и поддержку видеокарт. Здесь также можно найти tutorиалы по использованию модели внутри контейнера.

Чтобы сохранить этот образ у себя, копируем команду в окошке **Docker Pull Command** и запускаем контейнер. Подробнее о том, как это делать, можно найти в отдельной документации. Использование Docker несложно, хотя требует времени для привыкания. Но, разобравшись с этим, можно значительно упростить себе жизнь.

5. Как запускать LLM локально. Часть 2

Llama.cpp

Llama.cpp - реализация LLM ориентированная на производительность, обеспечивая быстрый способ работы с языковыми моделями.

С помощью Llama.cpp можно легко  модели локально, даже без GPU

Требование: модель в формате **.GGUF**

<https://github.com/ggerganov/llama.cpp>

zerocoder.ru



Портал Llama.CPP

Ещё один важный инструмент для локального запуска больших языковых моделей — это Llama CPP.

Действия

✓ Llama CPP — это инструмент для реализации больших языковых моделей, ориентированный на производительность и обеспечение быстрого способа работы с моделями. Этот инструмент написан на языке программирования C++. Он оптимизирован и работает значительно быстрее, чем Python.

Llama CPP позволяет запускать большие языковые модели даже на самых слабых компьютерах, без необходимости в видеокарте.

Посмотрим на tutorial по использованию Llama CPP, доступный на странице <https://github.com/ggerganov/llama.cpp> (<https://github.com/ggerganov/llama.cpp>).

Установка этого модуля несложная, особенно если на операционной системе Linux. На Mac это может быть немного сложнее, а на Windows ещё сложнее, но всё равно вполне выполнимо.

Для начала нужно скопировать репозиторий, затем, если на компьютере установлена операционная система Linux или Mac, запустить команду `make`; на ОС Windows нужно запустить файл `exe` и выполнить несколько дополнительных шагов — все они описаны в tutorialе на указанном сайте. После этого можно использовать большие языковые модели в командной строке, как при общении с ChatGPT, только локально.

Хотя проект называется Llama CPP, это не значит, что можно использовать только модели Llama. Поддерживаются любые модели в формате GGUF.

Настройка Kobold CPP

Рассмотрим настройку Llama CPP — Kobold CPP. Эта настройка предлагает удобный интерфейс, включающий чат-ассистента и возможность создавать своих агентов или использовать готовых.

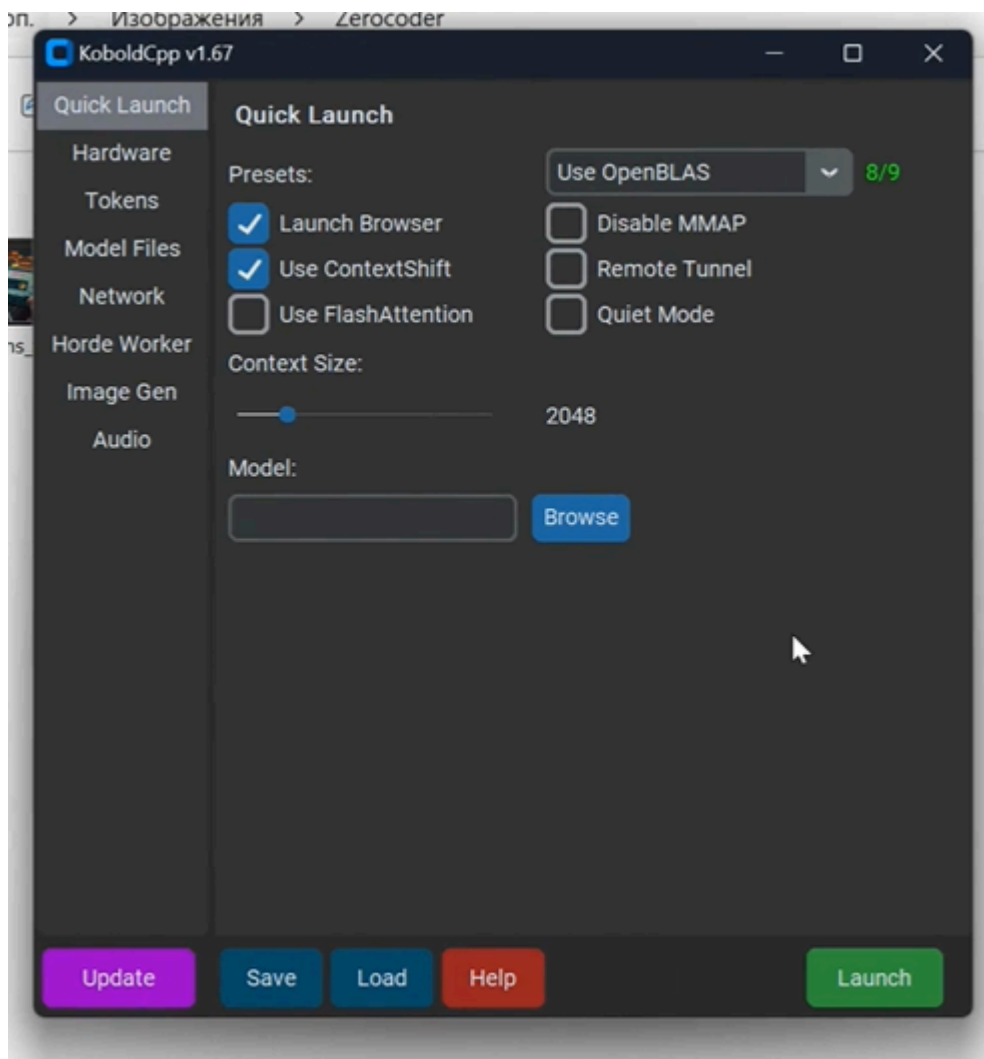
Перейдём по ссылке

<https://github.com/LostRuins/koboldcpp/releases>

(<https://github.com/LostRuins/koboldcpp/releases>) и посмотрим, что нужно сделать для установки. Здесь всё устроено гораздо проще, чем на Llama CPP. Скачиваем установочный файл, выбрав его из раздела **Assetes**. Если на компьютере установлена ОС Windows, скачиваем файл **exe**, выбирая версию в зависимости от наличия видеокарты. Если видеокарты нет, скачиваем файл с расширением **nocuda**. Если видеокарта есть, нужно определить версию драйвера CUDA и скачать соответствующий файл: CUDA 12 или CUDA 1150.

После скачивания запускаем файл **exe**, который при запуске потребует файл модели в формате GGUF. Поэтому перед запуском exe найдём модель в этом формате на HuggingFace. Есть много вариаций модели Llama — например, TinyLlama, которая содержит около миллиарда параметров. В поисковой строке на HuggingFace нужно найти TinyLlama gguf, чтобы у модели был подходящий формат.

Представим, что мы скачали файл **exe** и модель. Переходим в папку на компьютере, куда скачались файлы, и запускаем файл exe. В процессе запуска создаётся интерфейс.



Здесь можно запустить большую языковую модель, нажав на кнопку **Launch**. Далее в открывшемся окне проводника необходимо выбрать скачанный на компьютер файл модели. После этого в окне браузера по localhost-5001 откроется диалоговый интерфейс.

В этом интерфейсе можно начать новую сессию и задать вопрос модели. Например, попросим модель написать поэму о зерокодерах. При этом не стоит ожидать слишком многого от этой модели: у нее всего лишь миллиард параметров; к тому же, она сильно квантизована. Но, тем не менее, даже на слабом компьютере модель запускается и отвечает в режиме реального времени. Если скачать с Hugging Face большие модели в формате GGUF и использовать видеокарту, производительность будет гораздо выше.

Теперь рассмотрим, что ещё можно делать в этом интерфейсе запущенной модели. Во вкладке **Scenarios** можно выбрать различные ^{Действия} агенты с конкретными функциями. Здесь также через

вкладку **Settings** можно создавать свои сценарии, настраивать параметры генерации токенов, температуру и другие настройки.

Библиотека Langchain

✓ Langchain — это библиотека для создания приложений, использующих большие языковые модели.

Информация о Langchain

https://python.langchain.com/v0.1/docs/get_started/quickstart/
(https://python.langchain.com/v0.1/docs/get_started/quickstart/).

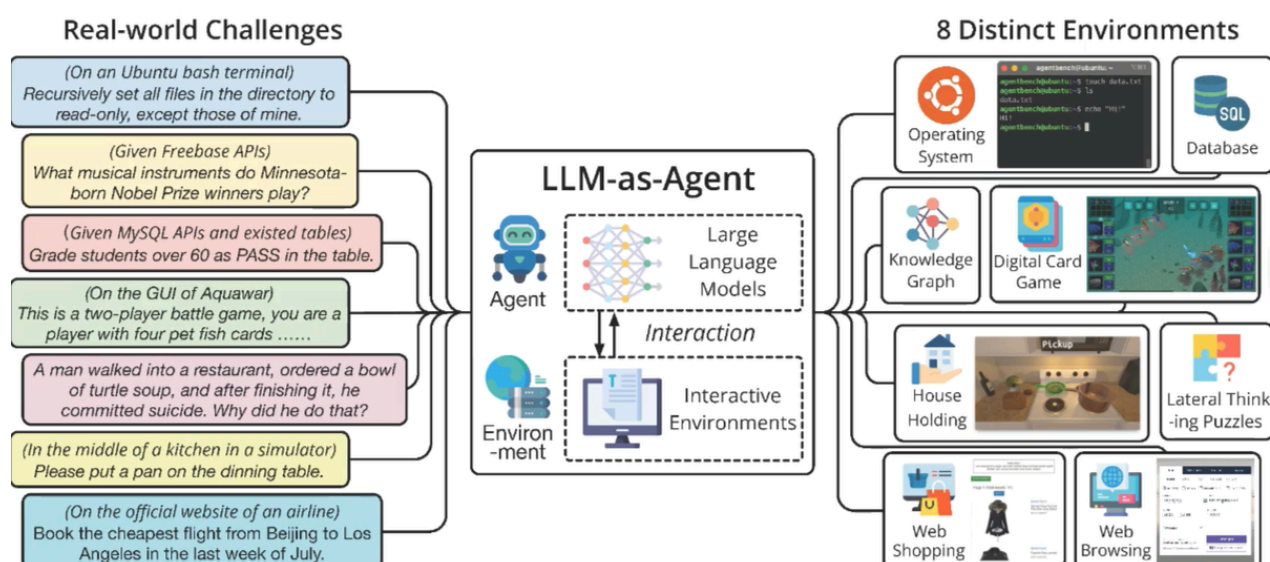
С помощью Langchain можно:

- создавать собственный API-сервер;
- обрабатывать цепочки запросов, что важно для естественного взаимодействия с пользователем. Langchain;
- работа с контекстом (с предыдущими запросами);
- создавать локальных мультимодельных ассистентов, комбинировать различные модели и оборачивать всё в готовые приложения.

Если просто запускать модель с HuggingFace, эти модели будут видеть только тот контекст, который мы им показываем в данный момент. Если спросить что-то у такой модели, получить ответ и затем попробовать уточнить этот ответ, прояснить детали, модель ничего не ответит, поскольку она нигде не хранит контекст. Если же необходимо обрабатывать именно цепочки запросов в более естественной интеракции с пользователем, лучше использовать LangChain.

Более детальную информацию о LangChain можно получить на их официальной странице, где содержится tutorial и вся необходимая документация.

✅ LLM агенты — это автономная система, где большая языковая модель является "мозгами", к которым подключены API и возможность их самостоятельно вызывать и работать с результатами этих вызовов. Агент может выходить в интернет, находить информацию, планировать, обращаться к файлам и базам данных, реализуя Retrieval Augmented Generation (RAG).



Есть несколько агентов, которыми можно пользоваться и запускать локально, например, BabyAGI — комбинация Llama с планировщиком и технологией RAG. Это ещё сырая технология, требующая доработки, но перспективная. В ближайшие месяцы агенты станут более функциональными и смогут выполнять множество задач.

Если необходимо создать своих агентов, очень пригодится фреймворк AutoGPT.

Оценка агентов — непростая задача, поскольку оценивать их можно на самых разных задачах, и сложно найти какую-то одну метрику, которая способна оценить LLM однозначно. Агенты выполняют самые разнообразные сложные цепочки задач. Для оценки агентов используются специальные бенчмарки, например, WebArena и AgentBench. Они помогут выбрать лучших агентов для ваших задач.

- AutoGPT [<https://github.com/Significant-Gravitas/AutoGPT> (<https://github.com/Significant-Gravitas/AutoGPT>)]
 - Фреймворк для создания своих агентов
- BabyAGI - [<https://github.com/yoheinakajima/babyagi> (<https://github.com/yoheinakajima/babyagi>)]
 - LLaMa + RAG + планировщик

Оценка агентов:

- AgentBench [<https://github.com/THUDM/AgentBench> (<https://github.com/THUDM/AgentBench>)]
- WebArena [<https://webarena.dev/> (<https://webarena.dev/>)]

💡 Информацию про Langchain и LLM агенты мы разберем более подробно в следующих уроках.

Основные выводы урока

1. Для оценки системных требований нужно знать размер модели, формат хранения весов и режим использования. Для инференса достаточно умножить количество весов на размер одного веса в памяти. Для обучения потребуется больше ресурсов.
2. Эмбединги могут решать множество простых локальных задач, таких как классификация, кластеризация и создание мультимодельных систем.
3. Простейшие модели можно запустить локально всего в пару кликов даже без GPU. Однако для создания более сложных приложений потребуется более широкий тулбокс.

Для понимания темы полезно изучить библиотеку Transformers от Hugging Face для лучшего понимания её работы. Также важно научиться использовать Docker для создания изолированных исполнительных сред. Если же нужно, чтобы модель учитывала контекст или создавала мультимодельную систему, полезно изучить работу с LangChain. Все это может сделать наши модели и агенты сильнее и конкурентоспособнее в различных бенчмарках и соревнованиях.

Действия

Это всё, что мы хотели обсудить в сегодняшнем уроке. До новых встреч!

Домашнее задание

1. Оценка системных требований для локального запуска модели

- **Выбор модели:** Выберите большую языковую модель, например, GPT-3. Модель можно выбрать на платформе Hugging Face или в других источниках.
- **Сбор информации о модели:** Найдите технические данные о модели. Это можно сделать через документацию, статьи или официальные ресурсы, такие как Hugging Face.
- **Определение параметров системы:** Для оценки системных требований вам нужно будет собрать информацию о следующих параметрах:
 - Объем памяти VRAM
 - Количество ядер CUDA/Stream Processors
 - Пропускная способность видеокарты
 - Энергопотребление
- Всю информацию оформите в документе Google Docs.
- В качестве домашнего задания пришлите ссылку на документ (не забудьте открыть доступ для чтения).

2. Инструменты для локального развертывания модели

- Внимательно ознакомьтесь с информацией обо всех инструментах, с помощью которых можно запускать модели локально.
- Сделай сравнительную таблицу всех инструментов: опиши минусы и плюсы каждого инструмента, для каких задач подходит тот или иной инструмент.
- Всю информацию оформи в виде таблицы в документе Google Документы.
- В качестве домашнего задания пришли ссылку на документ (не забудь открыть доступ для чтения).

На следующем занятии

Рассмотрим облачные решения для LLM.

Если у вас есть вопросы, задавайте их, пожалуйста, в учебном чате, а не в этой анкете, — так вы сможете быстрее получить ответ.

Урок

PE9.2 Локальное развертывание моделей

Понравился ли урок?

- ☐ Понравился, все было понятно
- ☒ Неплохо, но не все получилось/не все понятно
- ☐ Не понравился

Обратная связь по уроку (не обязательно)

Отправить

ЗАДАНИЕ

1. Оценка системных требований для локального запуска модели

- Выбор модели:

Действия

Выберите большую языковую модель, например, GPT-3. Модель можно выбрать на платформе Hugging Face или в других источниках.

- Сбор информации о модели:

Найдите технические данные о модели. Это можно сделать через документацию, статьи или официальные ресурсы, такие как Hugging Face.

- Определение параметров системы:

Для оценки системных требований вам нужно будет собрать информацию о следующих параметрах:

- Объем памяти VRAM
 - Количество ядер CUDA/Stream Processors
 - Пропускная способность видеокарты
 - Энергопотребление
- Вся информацию оформите в документе Google Docs.
- В качестве домашнего задания пришлите ссылку на документ (не забудьте открыть доступ для чтения).

2. Инструменты для локального развертывания модели

- Внимательно ознакомьтесь с информацией обо всех инструментах, с помощью которых можно запускать модели локально.
- Сделай сравнительную таблицу всех инструментов: опиши минусы и плюсы каждого инструмента, для каких задач подходит тот или иной инструмент.
- Вся информацию оформи в виде таблицы в документе Google Документы.
- В качестве домашнего задания пришли ссылку на документ (не забудь открыть доступ для чтения).



Ваш ответ



Прикрепить файлы

максимальный размер файла - 100мб

Отправить ответ

Сохранить как черновик

20 монет за выполнение этого урока

Предыдущий
урок

(<https://university.zerocoder.ru/pl/teach/control/lesson/view?id=338667559>)

Следующий урок

Действия